# Componente de Avaliação Projeto 1

# Arquitetura de Computadores Ano letivo: 2025/2026

**Nº 2178724** - Francisco Irineu Gonçalves Gouveia
**N º2146224** - Lucas José Do Nascimento Da Silva

## 1. Introdução

O presente relatório descreve o desenvolvimento e implementação de um processador básico de 8 bits , utilizando a linguagem de descrição de hardware VHDL. Este projeto enquadra-se na componente de avaliação prática da unidade curricular de Arquitetura de Computadores.

O desenvolvimento tecnológico atual exige uma compreensão profunda do funcionamento interno dos processadores. Nesse sentido, este projeto aborda desde a criação dos blocos lógicos fundamentais até à sua integração final. A modelação e simulação do circuito foram realizadas na ferramenta Vivado ML Edition (Vivado Simulator) , tendo como objetivo final a síntese e configuração física do hardware numa FPGA Artix-7, especificamente na placa de desenvolvimento NEXYS A7/4 DDR.

## 2. Objetivos

O projeto tem como principal objetivo a conceção de um processador capaz de executar um conjunto de instruções. Os objetivos específicos incluem:

- Desenhar e programar os módulos internos do processador (Unidade Aritmética e Lógica, Registos, Contador de Programa, Descodificador e Gestor de Periféricos) .
- Desenvolver a estrutura superior (Placa-Mãe ou Top-Level) recorrendo a uma modelação estrutural em VHDL, interligando o processador com a Memória de Instruções (ROM) e a Memória de Dados (RAM).
- Validar a arquitetura computacional através de simulações (Testbench), comprovando o correto funcionamento de todos os módulos.
- Preparar o mapeamento de portas de entrada e saída (PIN e POUT) para os periféricos físicos da FPGA.

# 3. Desenvolvimento

A arquitetura do sistema foi desenvolvida com base num design hierárquico e modular, o que facilita a deteção de erros e a manutenção do código. O sistema divide-se em duas componentes principais: o Processador (unidade de controlo) e a Placa-Mãe (integração).

## 3.1. O Processador

O núcleo do sistema foi implementado contendo os seguintes módulos essenciais:

- **Unidade Aritmética e Lógica (ALU):** Responsável por processar operandos de 8 bits em complemento para dois, executando operações aritméticas (soma, subtração), lógicas (AND, OR, XOR, etc.) e deslocamentos de bits (SHR, SHL) .
- **Registos e Memória Temporária:** O sistema possui dois registos de uso geral, o Registo A e o Registo B, selecionados através de multiplexers. Existe também um Registo de Flags que armazena os estados das operações de comparação da ALU para controlo de fluxos condicionais.
- **Flow control:** O Contador de Programa (PC) cria os endereços para a ROM. Um Multiplexer do PC avalia o estado das flags e decide se o sistema faz um incremento sequencial ou um salto na memória (instruções JMP, JZ, JL, etc.).
- **Descodificador:** O "cérebro" de controlo que interpreta o opcode de 5 bits vindo da ROM e ativa os sinais necessários (SEL_ALU, WR, ESCR_R, etc.) para garantir a execução da instrução no ciclo de relógio correto.
- **Gestor de Periféricos:** Gere a comunicação de 8 bits com o utilizador, lendo do sinal de entrada (PIN) e escrevendo no sinal de saída (POUT).

## 3.2. A Placa-Mãe (Integração Estrutural)

Ao nível estrutural superior (Motherboard), o Processador foi instanciado em conjunto com a ROM (que fornece palavras de 14 bits) e com a RAM (para persistência de dados). O mapeamento de portas (Port Map) assegurou a partilha do barramento de dados, endereços e relógio (CLK) de forma síncrona.

# Discussão dos Resultados

Para validar o sistema de forma rigorosa, utilizou-se o programa de testes fornecido na ROM (Tabela 9), executado num ambiente de simulação (*Testbench*). O programa continha três rotinas distintas de validação de hardware:

- **Teste de Aritmética e Ciclos:** Introduzindo o valor decimal "5" na entrada (PIN), o processador entrou com sucesso na rotina de soma cíclica. O sistema carregou o valor, fez somas sucessivas (ADD) na ALU e comprovou o funcionamento dos saltos condicionais iterativos (JL). A saída (POUT) apresentou a contagem exata até parar na condição final (50).
- **Teste de Lógica Numérica e Negativos:** Ao inserir o valor negativo "-16" na entrada, o sistema validou o salto para números menores que zero. A execução demonstrou que a máquina opera perfeitamente com a notação de complemento para dois, processando portas lógicas exclusivas (XOR) e enviando o valor absoluto (módulo) esperado para o exterior.
- **Teste de Acesso a Memória e Shifts (RAM):** Submetendo o sistema a um valor maior que 50 (ex: 60). O processador utilizou a RAM para armazenar e atualizar variáveis temporárias de contagem (ST e LD) , operou lógicas de isolamento de bits (AND) e efetuou deslocamentos corretos à direita (SHR). O contador devolveu com precisão o número de bits a "1" presentes no valor original (o valor 60, **00111100** em binário, resultou na saída exata de 4).

**Obs:** Estes resultados em forma de onda (ver Anexo A) provam que as ligações internas, a sincronização do relógio e a lógica de descodificação são funcionalmente íntegras.

# 5. Conclusão

O desenvolvimento deste projeto prático permitiu consolidar de forma clara os conceitos teóricos de arquitetura de computadores através da modelação em VHDL. A abordagem hierárquica demonstrou-se fundamental: ao abstrair a complexidade dos blocos individuais, a integração final na Placa-Mãe tornou-se num processo estritamente focado no encaminhamento de dados .

A simulação confirmou o cumprimento integral dos objetivos do projeto, com o processador a executar instruções que envolviam escrita em RAM, manipulação aritmética e lógicas bit a bit de acordo com as restrições síncronas de relógio impostas. O sistema encontra-se 100% funcional tanto nos testbenches como no FPGA Artix-7.

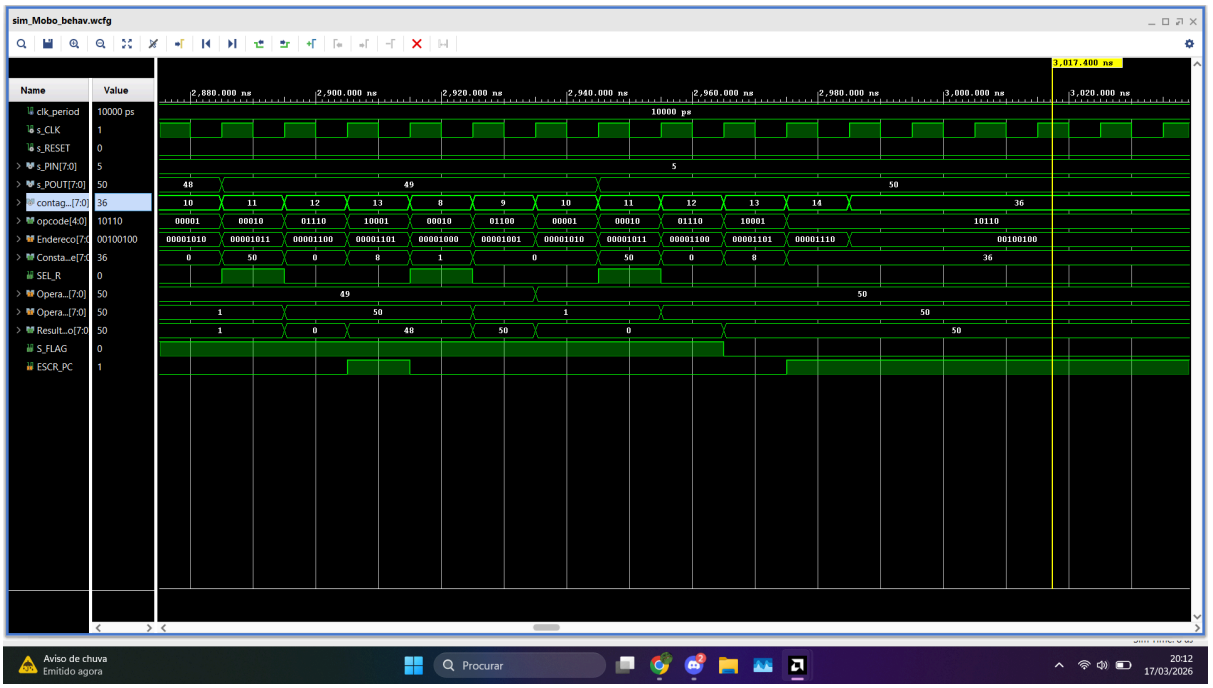
# 8. Bibliografia

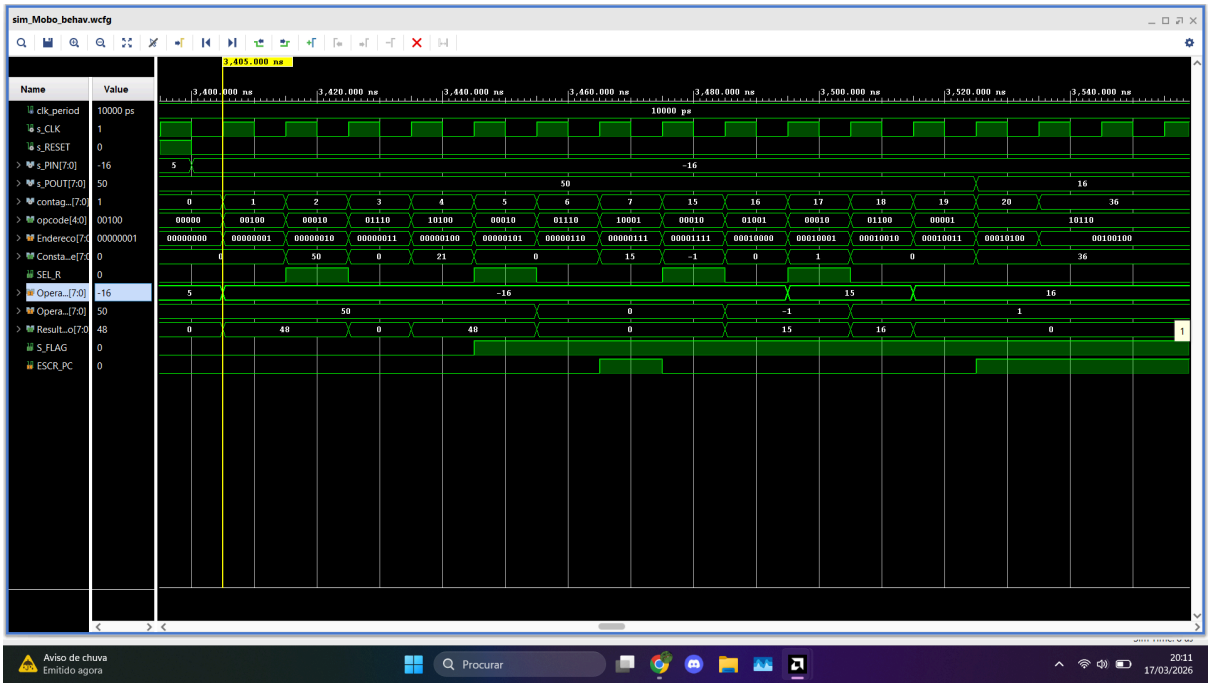
J. Delgado e C. Ribeiro, Arquitectura de Computadores, FCA, 2007 D. M. Harris and S. L. Harris, Digital Design and Computer Architecture, Elsevier MK, 2007

# Tabela 6:

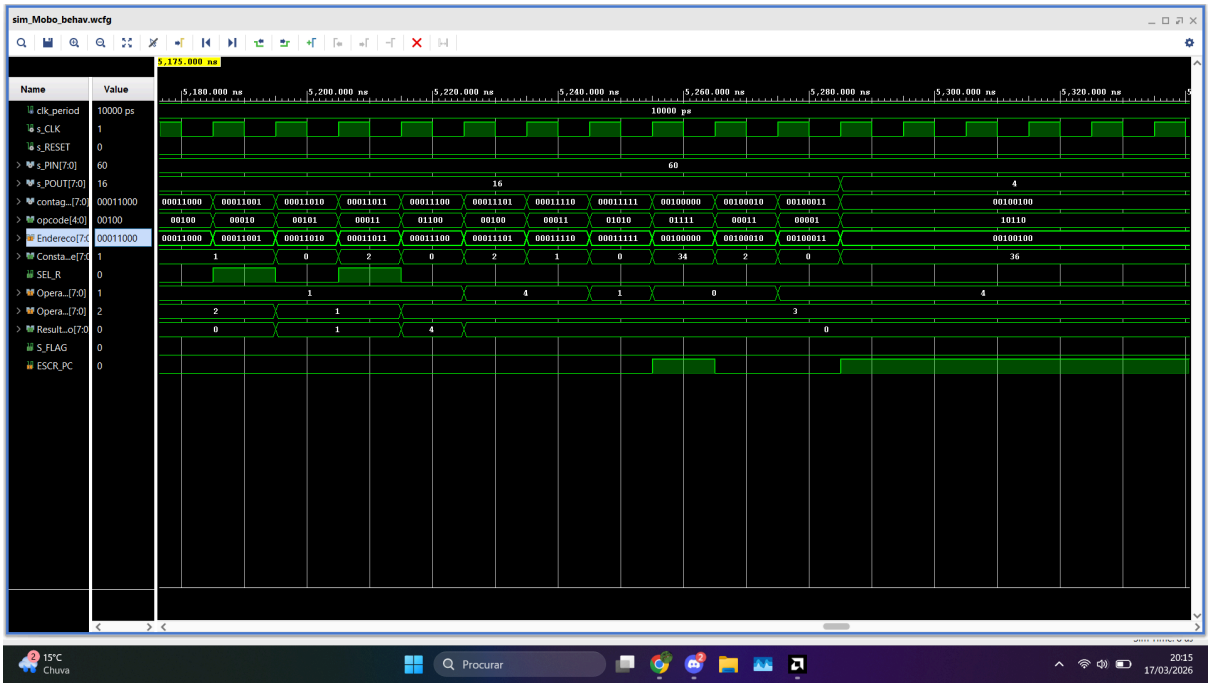
| Endereço | Instrução (Assembly) | Instrução (código máquina) |
|---|---|---|
| 00000000 | LDP RA | 00000 0 00000000 |
| 00000001 | ST [0], RA | 00100 0 00000000 |
| 00000010 | LD RB 50 | 00010 1 00110010 |
| 00000011 | CMP RA, RB | 01110 0 00000000 |
| 00000100 | JGE 21 | 10100 0 00010101 |
| 00000101 | LD RB, 0 | 00010 1 00000000 |
| 00000110 | CMP RA, RB | 01110 0 00000000 |
| 00000111 | JL RA, 15 | 10001 0 00001111 |
| 00001000 | LD RB, 1 | 00010 1 00000001 |
| 00001001 | ADD RA, RB | 01100 0 00000000 |
| 00001010 | STP RA | 00001 0 00000000 |
| 00001011 | LD RB, 50 | 00010 1 00110010 |
| 00001100 | CMP RA, RB | 01110 0 00000000 |
| 00001101 | JL 8 | 10001 0 00001000 |
| 00001110 | JMP 36 | 10110 0 00100100 |
| 00001111 | LD RB, -1 | 00010 1 11111111 |
| 00010000 | XOR RA, RB | 01001 0 00000000 |
| 00010001 | LD RB, 1 | 00010 1 00000001 |
| 00010010 | ADD RA, RB | 01100 0 00000000 |
| 00010011 | STP RA | 00001 0 00000000 |
| 00010100 | JMP 36 | 10110 0 00100100 |
| 00010101 | LD RA, 0 | 00010 0 00000000 |
| 00010110 | ST [2], RA | 00100 0 00000010 |
| 00010111 | LD RA, [0] | 00011 0 00000000 |
| 00011000 | ST [1], RA | 00100 0 00000001 |
| 00011001 | LD RB, 1 | 00010 1 00000001 |
| 00011010 | AND RA, RB | 00101 0 00000000 |
| 00011011 | LD RB, [2] | 00011 1 00000010 |
| 00011100 | ADD RA, RB | 01100 0 00000000 |
| 00011101 | ST [2], RA | 00100 0 00000010 |
| 00011110 | LD RA, [1] | 00011 0 00000001 |
| 00011111 | SHR RA | 01010 0 00000000 |
| 00100000 | JZ 34 | 01111 0 00100010 |
| 00100001 | JMP 24 | 10110 0 00011000 |
| 00100010 | LD RA, [2] | 00011 0 00000010 |
| 00100011 | STP RA | 00001 0 00000000 |
| 00100100 | JMP 36 | 10110 0 00100100 |

# 7.Anexo A


sim_Mobo_behav.wcfg
Name
Value
clk_period
10000 ps
s_CLK
s_RESET
s_PIN[7:0]
s_POUT[7:0]
contag...[7:0]
opcode[4:0]
Endereco[7:
Consta...e[7:
SEL_R
Opera...[7:0]
Opera...[7:0]
Result...o[7:0]
S_FLAG
ESCR_PC
3,017.400 ns
Aviso de chuva
Emitido agora
Procurar
20:12
17/03/2026




Aviso de chuva
Emitido agora
Procurar
20:11
17/03/2026


sim_Mobo_behav.wcfg
Name
Value
clk_period 10000 ps
s_CLK 1
s_RESET 0
s_PIN[7:0] 60
s_POUT[7:0] 16
contag...[7:0] 00011000
opcode[4:0] 00100
Endereco[7:0] 00011000
Consta...e[7:0] 1
SEL_R 0
Opera...[7:0] 1
Opera...[7:0] 2
Result...o[7:0] 0
S_FLAG 0
ESCR_PC 0
5,175.000 ns
5,180.000 ns
5,200.000 ns
5,220.000 ns
5,240.000 ns
5,260.000 ns
5,280.000 ns
5,300.000 ns
5,320.000 ns
10000 ps
60
16
4
00011000 00011001 00011010 00011011 00011100 00011101 00011110 00011111 00100000 00100010 00100011 00100100
00100 00010 00101 00011 01100 00100 00011 01010 01111 00011 00001 10110
00011000 00011001 00011010 00011011 00011100 00011101 00011110 00011111 00100000 00100010 00100011 00100100
1 0 2 0 2 1 0 34 2 0 36
1 4 1 0 4
2 1 3
0 1 4 0
15°C Chuva
Procurar
20:15 17/03/2026

# 8. Anexo B

## Motherboard:

```
library IEEE;

use IEEE.STD_LOGIC_1164.ALL;


entity Mobo is

   Port (

        CLK   : in  STD_LOGIC;

        RESET : in  STD_LOGIC;

        PIN   : in  STD_LOGIC_VECTOR (7 downto 0);

        POUT  : out STD_LOGIC_VECTOR (7 downto 0)

       );

end Mobo;


architecture Behavioral of Mobo is


   component Processador is

      Port(

         CLK       : in  STD_LOGIC;

         RESET     : in  STD_LOGIC;

         PIN       : in  STD_LOGIC_VECTOR (7 downto 0);

         POUT      : out STD_LOGIC_VECTOR (7 downto 0);

         opcode    : in  STD_LOGIC_VECTOR (4 downto 0);

         Constante : in  STD_LOGIC_VECTOR (7 downto 0);

         SEL_R_in  : in  STD_LOGIC;

         Dados_M   : in  STD_LOGIC_VECTOR (7 downto 0);

         WR        : out STD_LOGIC;

         Endereco  : out STD_LOGIC_VECTOR (7 downto 0);

         Dados_W   : out STD_LOGIC_VECTOR (7 downto 0)

      );

   end component;
```

```
    component RAM is
        Port (
            DadosIN  : in  STD_LOGIC_VECTOR (7 downto 0);
            Endereco : in  STD_LOGIC_VECTOR (7 downto 0);
            WR       : in  STD_LOGIC;
            CLK      : in  STD_LOGIC;
            DadosOUT : out STD_LOGIC_VECTOR (7 downto 0)
        );
    end component;

    component ROM_Memoria_Instrucoes is
        Port (
            Endereco  : in  STD_LOGIC_VECTOR (7 downto 0);
            opcode    : out STD_LOGIC_VECTOR (4 downto 0);
            SEL_R     : out STD_LOGIC;
            Constante : out STD_LOGIC_VECTOR (7 downto 0)
        );
    end component;

    -- Fios de ligação interna (sinais)
    signal s_Endereco  : std_logic_vector (7 downto 0);
    signal s_opcode    : std_logic_vector (4 downto 0);
    signal s_Constante : std_logic_vector (7 downto 0);
    signal s_SEL_R     : std_logic;

    signal s_WR        : std_logic;
    signal s_Dados_M   : std_logic_vector (7 downto 0);
    signal s_Dados_W   : std_logic_vector (7 downto 0);

begin

    Inst_Processador : Processador
        port map (
```

```
        CLK       => CLK,
        RESET     => RESET,
        PIN       => PIN,
        POUT      => POUT,
        opcode    => s_opcode,
        Constante => s_Constante,
        SEL_R_in  => s_SEL_R,
        Dados_M   => s_Dados_M,
        WR        => s_WR,
        Endereco  => s_Endereco,
        Dados_W   => s_Dados_W
    );

  Inst_RAM : RAM
    port map (
        DadosIN  => s_Dados_W,
        Endereco => s_Constante,
        WR       => s_WR,
        CLK      => CLK,
        DadosOUT => s_Dados_M
    );

  Inst_ROM : ROM_Memoria_Instrucoes
    port map (
        Endereco  => s_Endereco,
        opcode    => s_opcode,
        SEL_R     => s_SEL_R,
        Constante => s_Constante
    );

end Behavioral;
```

# **RAM**:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity RAM is
   Port (
        DadosIN  : in  STD_LOGIC_VECTOR (7 downto 0);
        Endereco : in  STD_LOGIC_VECTOR (7 downto 0);
        WR       : in  STD_LOGIC;
        CLK      : in  STD_LOGIC;
        DadosOUT : out STD_LOGIC_VECTOR (7 downto 0)
       );
end RAM;

architecture Behavioral of RAM is

   type memoria is array (0 to 255) of std_logic_vector (7 downto 0);

   -- Inicialização da RAM toda a zeros para evitar o estado 'U' no simulador
   signal MemEndereco : memoria := (others => (others => '0'));

begin

   -- Lógica de Escrita Síncrona (Apenas na subida do CLK e se WR = '1')
   process(CLK)
   begin
      if rising_edge(CLK) then
         if WR = '1' then
            MemEndereco(to_integer(unsigned(Endereco))) <= DadosIN;
         end if;
      end if;
```

```
    end process;


    -- Lógica de Leitura Assíncrona (Fora do processo)

    -- Quando WR = '0' lê a RAM, caso contrário coloca a zeros

    DadosOUT <= MemEndereco(to_integer(unsigned(Endereco))) when WR = '0' else (others => '0');


end Behavioral;
```

# ROM:

```
----------------------------------------------------------------------------------
-- Company:
-- Engineer:
--
-- Create Date: 05.03.2026 12:59:34
-- Design Name:
-- Module Name: ROM_Memoria_Instrucoes - Behavioral
-- Project Name:
-- Target Devices:
-- Tool Versions:
-- Description:
--
-- Dependencies:
--
-- Revision:
-- Revision 0.01 - File Created
-- Additional Comments:
--
----------------------------------------------------------------------------------


library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

entity ROM_Memoria_Instrucoes is
    Port ( Endereco : in STD_LOGIC_VECTOR (7 downto 0);
           opcode : out STD_LOGIC_VECTOR (4 downto 0);
           SEL_R : out STD_LOGIC;
           Constante : out STD_LOGIC_VECTOR (7 downto 0)
           );
end ROM_Memoria_Instrucoes;

architecture Behavioral of ROM_Memoria_Instrucoes is

    signal instrucao : std_logic_vector(13 downto 0);
```

```
begin
--processo que define a instrucao
   process(Endereco)
   begin
      case Endereco is
         when "00000000" => instrucao <= "00000000000000"; -- 0000: OP LDP Ri
         when "00000001" => instrucao <= "00100000000000"; -- 1:  ST [0], RA
         when "00000010" => instrucao <= "00010100110010"; -- 2:  LD RB, 50
         when "00000011" => instrucao <= "01110000000000"; -- 3:  CMP RA, RB
         when "00000100" => instrucao <= "10100000010101"; -- 4:  JGE 21
         when "00000101" => instrucao <= "00010100000000"; -- 5:  LD RB, 0
         when "00000110" => instrucao <= "01110000000000"; -- 6:  CMP RA, RB
         when "00000111" => instrucao <= "10001000001111"; -- 7:  JL 15
         when "00001000" => instrucao <= "00010100000001"; -- 8:  LD RB, 1
         when "00001001" => instrucao <= "01100000000000"; -- 9:  ADD RA, RB
         when "00001010" => instrucao <= "00001000000000"; -- 10: STP RA
         when "00001011" => instrucao <= "00010100110010"; -- 11: LD RB, 50
         when "00001100" => instrucao <= "01110000000000"; -- 12: CMP RA, RB
         when "00001101" => instrucao <= "10001000001000"; -- 13: JL 8
         when "00001110" => instrucao <= "10110000100100"; -- 14: JMP 36
         when "00001111" => instrucao <= "00010111111111"; -- 15: LD RB, -1
         when "00010000" => instrucao <= "01001000000000"; -- 16: XOR RA, RB
         when "00010001" => instrucao <= "00010100000001"; -- 17: LD RB, 1
         when "00010010" => instrucao <= "01100000000000"; -- 18: ADD RA, RB
         when "00010011" => instrucao <= "00001000000000"; -- 19: STP RA
         when "00010100" => instrucao <= "10110000100100"; -- 20: JMP 36
         when "00010101" => instrucao <= "00010000000000"; -- 21: LD RA, 0
         when "00010110" => instrucao <= "00100000000010"; -- 22: ST [2], RA
         when "00010111" => instrucao <= "00011000000000"; -- 23: LD RA, [0]
         when "00011000" => instrucao <= "00100000000001"; -- 24: ST [1], RA
         when "00011001" => instrucao <= "00010100000001"; -- 25: LD RB, 1
         when "00011010" => instrucao <= "00101000000000"; -- 26: AND RA, RB
         when "00011011" => instrucao <= "00011100000010"; -- 27: LD RB, [2]
         when "00011100" => instrucao <= "01100000000000"; -- 28: ADD RA, RB
         when "00011101" => instrucao <= "00100000000010"; -- 29: ST [2], RA
         when "00011110" => instrucao <= "00011000000001"; -- 30: LD RA, [1]
         when "00011111" => instrucao <= "01010000000000"; -- 31: SHR RA
         when "00100000" => instrucao <= "01111000100010"; -- 32: JZ RA, 34
         when "00100001" => instrucao <= "10110000011000"; -- 33: JMP 24
         when "00100010" => instrucao <= "00011000000010"; -- 34: LD RA, [2]
         when "00100011" => instrucao <= "00001000000000"; -- 35: STP RA
         when "00100100" => instrucao <= "10110000100100"; -- 36: JMP 36
         when others => instrucao <= "XXXXXXXXXXXXXX"; -- mete o resto a zeros
      end case;
   end process;
   opcode    <= instrucao(13 downto 9); -- 5 bits de Opcode
   SEL_R     <= instrucao(8);           -- 1 bit de Seleção de Registo
   Constante <= instrucao(7 downto 0);  -- 8 bits de Constante
end Behavioral;
```

# PROCESSADOR:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity Processador is
   Port (
        CLK       : in  STD_LOGIC;
        RESET     : in  STD_LOGIC;
        PIN       : in  STD_LOGIC_VECTOR (7 downto 0);
        POUT      : out STD_LOGIC_VECTOR (7 downto 0);
        opcode    : in  STD_LOGIC_VECTOR (4 downto 0);
        Constante : in  STD_LOGIC_VECTOR (7 downto 0);
        SEL_R_in  : in  STD_LOGIC;
        Dados_M   : in  STD_LOGIC_VECTOR (7 downto 0);
        WR        : out STD_LOGIC;
        Endereco  : out STD_LOGIC_VECTOR (7 downto 0);
        Dados_W   : out STD_LOGIC_VECTOR (7 downto 0)
      );
end Processador;

architecture Behavioral of Processador is

   component ALU is
      Port (
           Operando1 : in  STD_LOGIC_VECTOR (7 downto 0);
           Operando2 : in  STD_LOGIC_VECTOR (7 downto 0);
           SEL_ALU   : in  STD_LOGIC_VECTOR (3 downto 0);
           Resultado : out STD_LOGIC_VECTOR (7 downto 0);
           E_FLAG    : out STD_LOGIC_VECTOR (4 downto 0)
          );
   end component;

   component Gestor_Perifericos is
      Port (
           CLK       : in  STD_LOGIC;
           ESCR_P    : in  STD_LOGIC;
           PIN       : in  STD_LOGIC_VECTOR (7 downto 0);
           Operando1 : in  STD_LOGIC_VECTOR (7 downto 0);
           Dados_IN  : out STD_LOGIC_VECTOR (7 downto 0);
           POUT      : out STD_LOGIC_VECTOR (7 downto 0)
          );
   end component;

   component MUX_Reg is
      Port (
           resultado  : in  std_logic_vector (7 downto 0);
           Dados_IN   : in  std_logic_vector (7 downto 0);
           Dados_Mem  : in  std_logic_vector (7 downto 0);
           Constante  : in  std_logic_vector (7 downto 0);
           SEL_Data   : in  std_logic_vector (1 downto 0);
           Dados_Regs : out std_logic_vector (7 downto 0)
          );
   end component;

   component ProgramCounter is
      Port (
           Constante : in  STD_LOGIC_VECTOR (7 downto 0);
           ESCR_PC   : in  STD_LOGIC;
           CLK       : in  STD_LOGIC;
           Reset     : in  STD_LOGIC;
           Endereco  : out STD_LOGIC_VECTOR (7 downto 0)
          );
   end component;

   component REGISTO_FLAGS is
      Port (
           CLK    : in  STD_LOGIC;
           E_FLAG : in  STD_LOGIC_VECTOR (4 downto 0);
           ESCR_F : in  STD_LOGIC;
```

```
            SEL_F  : in  STD_LOGIC_VECTOR (2 downto 0);
            S_FLAG : out STD_LOGIC
          );
    end component;

    component Registos_A_e_B is
      Port (
            CLK       : in  STD_LOGIC;
            ESCR_R    : in  STD_LOGIC;
            Dados_R   : in  STD_LOGIC_VECTOR (7 downto 0);
            SEL_R     : in  STD_LOGIC;
            Operando1 : out STD_LOGIC_VECTOR (7 downto 0);
            Operando2 : out STD_LOGIC_VECTOR (7 downto 0)
          );
    end component;

    component Descodificacao_ROM is
      Port (
            Opcode   : in  STD_LOGIC_VECTOR (4 downto 0);
            SEL_ALU  : out STD_LOGIC_VECTOR (3 downto 0);
            ESCR_P   : out STD_LOGIC;
            SEL_DATA : out STD_LOGIC_VECTOR (1 downto 0);
            ESCR_R   : out STD_LOGIC;
            WR       : out STD_LOGIC;
            SEL_PC   : out STD_LOGIC_VECTOR (2 downto 0);
            ESCR_F   : out STD_LOGIC;
            SEL_F    : out STD_LOGIC_VECTOR (2 downto 0)
          );
    end component;

    component MUX_PC is
      Port (
            S_FLAG    : in  std_logic;
            Operando1 : in  std_logic_vector (7 downto 0);
            SEL_PC    : in  std_logic_vector (2 downto 0);
            ESCR_PC   : out std_logic
          );
    end component;

    -- Sinais Internos
    signal s_Resultado : std_logic_vector(7 downto 0);
    signal s_Operando1 : std_logic_vector(7 downto 0);
    signal s_Operando2 : std_logic_vector(7 downto 0);
    signal s_Dados_IN  : std_logic_vector(7 downto 0);
    signal s_Dados_R   : std_logic_vector(7 downto 0);
    signal s_E_FLAG    : std_logic_vector(4 downto 0);
    signal s_S_FLAG    : std_logic;
    signal s_ESCR_PC   : std_logic;

    -- Fios do Descodificador da ROM
    signal s_SEL_PC : std_logic_vector(2 downto 0);
    signal s_SEL_F  : std_logic_vector(2 downto 0);
    signal s_ESCR_F : std_logic;
    signal s_SEL_ALU: std_logic_vector (3 downto 0);
    signal s_ESCR_R : std_logic;
    signal s_SEL_Data: std_logic_vector(1 downto 0);
    signal s_ESCR_P : std_logic;

begin

    Inst_ALU : ALU
      port map(
              Operando1 => s_Operando1,
              Operando2 => s_Operando2,
              SEL_ALU   => s_SEL_ALU,
```

```
            Resultado => s_Resultado,
            E_FLAG    => s_E_FLAG
           );

Inst_Gestor_Perifericos : Gestor_Perifericos
   port map (
            CLK       => CLK,
            ESCR_P    => s_ESCR_P,
            PIN       => PIN,
            Operando1 => s_Operando1,
            Dados_IN  => s_Dados_IN,
            POUT      => POUT
           );

Inst_MUX_Reg : MUX_Reg
   port map (
            resultado  => s_Resultado,
            Dados_IN   => s_Dados_IN,
            Dados_Mem  => Dados_M,
            Constante  => Constante,
            SEL_Data   => s_SEL_Data,
            Dados_Regs => s_Dados_R
           );

Inst_PC : ProgramCounter
   port map (
            Constante => Constante,
            ESCR_PC   => s_ESCR_PC,
            CLK       => CLK,
            Reset     => Reset,
            Endereco  => Endereco
           );

Inst_RF : REGISTO_FLAGS
   port map (
            CLK    => CLK,
            E_FLAG => s_E_FLAG,
            ESCR_F => s_ESCR_F,
            SEL_F  => s_SEL_F,
            S_FLAG => s_S_FLAG
           );

Inst_R_AB : Registos_A_e_B
   port map (
            CLK       => CLK,
            ESCR_R    => s_ESCR_R,
            Dados_R   => s_Dados_R,
            Sel_R     => SEL_R_in,
            Operando1 => s_Operando1,
            Operando2 => s_Operando2
           );

Inst_DROM : Descodificacao_ROM
   port map (
            Opcode   => opcode,
            SEL_ALU  => s_SEL_ALU,
            ESCR_P   => s_ESCR_P,
            SEL_DATA => s_SEL_Data,
            ESCR_R   => s_ESCR_R,
            WR       => WR,
            SEL_PC   => s_SEL_PC,
            ESCR_F   => s_ESCR_F,
            SEL_F    => s_SEL_F
           );
```

```
   Inst_MUXPC : MUX_PC
      port map (
               S_FLAG    => s_S_FLAG,
               Operando1 => s_Operando1,
               SEL_PC    => s_SEL_PC,
               ESCR_PC   => s_ESCR_PC
              );

   Dados_W <= s_Operando1;

end Behavioral;
```

# ALU:

```
----------------------------------------------------------------------------------
-- Company:
-- Engineer:
--
-- Create Date: 03.03.2026 18:43:00
-- Design Name:
-- Module Name: ALU - Behavioral
-- Project Name:
-- Target Devices:
-- Tool Versions:
-- Description:
--
-- Dependencies:
--
-- Revision:
-- Revision 0.01 - File Created
-- Additional Comments:
--
----------------------------------------------------------------------------------


library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

entity ALU is
   Port ( Operando1 : in STD_LOGIC_VECTOR (7 downto 0);
          Operando2 : in STD_LOGIC_VECTOR (7 downto 0);
          SEL_ALU : in STD_LOGIC_VECTOR (3 downto 0);
          Resultado : out STD_LOGIC_VECTOR (7 downto 0);
          E_FLAG : out STD_LOGIC_VECTOR (4 downto 0));
end ALU;

architecture Behavioral of ALU is

begin
   process(Operando1, Operando2, SEL_ALU)
   begin
   Resultado <= (others => '0');
   E_FLAG <= (others => '0');
      case SEL_ALU is
            when "0000" => Resultado <= (Operando1 and Operando2);
            when "0001" => Resultado <= (Operando1 or Operando2);
```

```
            when "0010" => Resultado <= (Operando1 nand Operando2);
            when "0011" => Resultado <= (Operando1 nor Operando2);
            when "0100" => Resultado <= (Operando1 xor Operando2);
            when "0101" => Resultado <= std_logic_vector(shift_right(unsigned(Operando1), 1));--bitshift para a direita
            when "0110" => Resultado <= std_logic_vector(shift_left(unsigned(Operando1), 1));--bitshift para a esquerda
            when "0111" => Resultado <= std_logic_vector(signed(Operando1) + signed(Operando2));
            when "1000" => Resultado <= std_logic_vector(signed(Operando1) - signed(Operando2));
            when "1001" =>
               if (signed(Operando1) <  signed(Operando2)) then E_FLAG(0) <= '1'; end if;
               if (signed(Operando1) <= signed(Operando2)) then E_FLAG(1) <= '1'; end if;
               if (signed(Operando1) =  signed(Operando2)) then E_FLAG(2) <= '1'; end if;
               if (signed(Operando1) >= signed(Operando2)) then E_FLAG(3) <= '1'; end if;
               if (signed(Operando1) >  signed(Operando2)) then E_FLAG(4) <= '1'; end if;
            when others => Resultado <= (others => '0');
         end case;
   end process;
end Behavioral;
```

# GESTOR DE PERIFERICOS:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Gestor_Perifericos is
   Port (
         CLK       : in  STD_LOGIC;
         ESCR_P    : in  STD_LOGIC;
         PIN       : in  STD_LOGIC_VECTOR (7 downto 0);
         Operando1 : in  STD_LOGIC_VECTOR (7 downto 0);
         Dados_IN  : out STD_LOGIC_VECTOR (7 downto 0);
         POUT      : out STD_LOGIC_VECTOR (7 downto 0)
       );
end Gestor_Perifericos;

architecture Behavioral of Gestor_Perifericos is
begin

   Dados_IN <= PIN when ESCR_P = '0' else (others => '0');

   process(CLK)
   begin
      if rising_edge(CLK) then
         if ESCR_P = '1' then
            POUT <= Operando1;
         end if;
      end if;
   end process;

end Behavioral;
```

# MUX_REGISTOS:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;
```

```
entity MUX_Reg is
  Port (
    resultado : in std_logic_vector (7 downto 0);
    Dados_IN : in std_logic_vector (7 downto 0);
    Dados_Mem : in std_logic_vector (7 downto 0);
    Constante : in std_logic_vector (7 downto 0);
    SEL_Data : in std_logic_vector (1 downto 0);
    Dados_Regs : out std_logic_vector (7 downto 0)
    );
end MUX_Reg;

architecture Behavioral of MUX_Reg is
begin
    process(SEL_Data, resultado, Dados_IN, Dados_Mem, Constante)
    begin
        case SEL_Data is
            when "00" => Dados_Regs <= resultado;
            when "01" => Dados_Regs <= Dados_IN;
            when "10" => Dados_Regs <= Dados_Mem;
            when "11" => Dados_Regs <= Constante;
            when others => Dados_Regs <= (others => '0');
        end case;
    end process;
end Behavioral;
```

# PROGRAMCOUNTER:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity ProgramCounter is
    Port (
        Constante : in  STD_LOGIC_VECTOR (7 downto 0);
        ESCR_PC   : in  STD_LOGIC;
        CLK       : in  STD_LOGIC;
        Reset     : in  STD_LOGIC;
        Endereco  : out STD_LOGIC_VECTOR (7 downto 0)
    );
end ProgramCounter;

architecture Behavioral of ProgramCounter is

    signal contagem : unsigned(7 downto 0);

begin

    process(CLK)
    begin
        if rising_edge(CLK) then
            if Reset = '1' then
                contagem <= (others => '0');
            elsif ESCR_PC = '1' then
                contagem <= unsigned(Constante);
            else
                contagem <= contagem + 1;
            end if;
        end if;
    end process;

    Endereco <= std_logic_vector(contagem);

end Behavioral;
```

# REGISTO DE FLAGS:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity REGISTO_FLAGS is
   Port (
      CLK    : in STD_LOGIC;
      E_FLAG : in STD_LOGIC_VECTOR (4 downto 0);
      ESCR_F : in STD_LOGIC;
      SEL_F  : in STD_LOGIC_VECTOR (2 downto 0);
      S_FLAG : out STD_LOGIC
   );
end REGISTO_FLAGS;

architecture Behavioral of REGISTO_FLAGS is

   signal registo_interno : STD_LOGIC_VECTOR(4 downto 0) := (others => '0');

begin

   process(CLK)
   begin
      if rising_edge(CLK) then
         if ESCR_F = '1' then
            registo_interno <= E_FLAG;
         end if;
      end if;
   end process;

   process(SEL_F, registo_interno)
   begin
      case SEL_F is
         when "000" => S_FLAG <= registo_interno(0); -- (<)
         when "001" => S_FLAG <= registo_interno(1); -- (<=)
         when "010" => S_FLAG <= registo_interno(2); -- (=)
         when "011" => S_FLAG <= registo_interno(3); -- (>=)
         when "100" => S_FLAG <= registo_interno(4); -- (>)
         when others => S_FLAG <= '0';
      end case;
   end process;

end Behavioral;
```

# REGISTO_A_B:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Registos_A_e_B is
   Port (
      CLK       : in  STD_LOGIC;
      ESCR_R    : in  STD_LOGIC;
      Dados_R   : in  STD_LOGIC_VECTOR (7 downto 0);
      SEL_R     : in  STD_LOGIC;
      Operando1 : out STD_LOGIC_VECTOR (7 downto 0);
      Operando2 : out STD_LOGIC_VECTOR (7 downto 0)
   );
end Registos_A_e_B;

architecture Behavioral of Registos_A_e_B is

   signal R_a : std_logic_vector (7 downto 0) := (others => '0');
   signal R_b : std_logic_vector (7 downto 0) := (others => '0');

begin
   process(CLK)
   begin
      if rising_edge(CLK) then
         if ESCR_R = '1' then
            if SEL_R = '1' then
               R_b <= Dados_R;
            else
               R_a <= Dados_R;
            end if;
         end if;
      end if;
   end process;

   Operando1 <= R_a;
   Operando2 <= R_b;

end Behavioral;
```

# DESCODIFICACAO ROM:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Descodificacao_ROM is
   Port (
         Opcode   : in  STD_LOGIC_VECTOR (4 downto 0);
         SEL_ALU  : out STD_LOGIC_VECTOR (3 downto 0);
         ESCR_P   : out STD_LOGIC;
         SEL_DATA : out STD_LOGIC_VECTOR (1 downto 0);
         ESCR_R   : out STD_LOGIC;
         WR       : out STD_LOGIC;
         SEL_PC   : out STD_LOGIC_VECTOR (2 downto 0);
         ESCR_F   : out STD_LOGIC;
         SEL_F    : out STD_LOGIC_VECTOR (2 downto 0)
   );
end Descodificacao_ROM;

architecture Behavioral of Descodificacao_ROM is

   signal controlo : std_logic_vector (15 downto 0);

begin

   process(Opcode)
```

```
    begin
      case Opcode is
        --                          |SEL_ALU|ESCR_p|SEL_DATA|ESCR_R|WR|SEL_PC|ESCR_F|SEL_F|
        -- Perifericos                |      |     |       |     | |     |     |    |
        when "00000" => controlo <= "0000" & '0' &  "01"  & '1' & '0' & "000" & '0' & "000"; -- LDP Ri
        when "00001" => controlo <= "0000" & '1' &  "00"  & '0' & '0' & "000" & '0' & "000"; -- STP RA

        -- Leitura e Escrita
        when "00010" => controlo <= "0000" & '0' &  "11"  & '1' & '0' & "000" & '0' & "000"; -- LD Ri, constante
        when "00011" => controlo <= "0000" & '0' &  "10"  & '1' & '0' & "000" & '0' & "000"; -- LD Ri, [constante]
        when "00100" => controlo <= "0000" & '0' &  "00"  & '0' & '1' & "000" & '0' & "000"; -- ST [constante], RA

        -- Lógica e Aritmética
        when "00101" => controlo <= "0000" & '0' &  "00"  & '1' & '0' & "000" & '0' & "000"; -- AND RA, RB
        when "00110" => controlo <= "0001" & '0' &  "00"  & '1' & '0' & "000" & '0' & "000"; -- OR RA, RB
        when "00111" => controlo <= "0010" & '0' &  "00"  & '1' & '0' & "000" & '0' & "000"; -- NAND RA, RB
        when "01000" => controlo <= "0011" & '0' &  "00"  & '1' & '0' & "000" & '0' & "000"; -- NOR RA, RB
        when "01001" => controlo <= "0100" & '0' &  "00"  & '1' & '0' & "000" & '0' & "000"; -- XOR RA, RB
        when "01010" => controlo <= "0101" & '0' &  "00"  & '1' & '0' & "000" & '0' & "000"; -- SHR RA
        when "01011" => controlo <= "0110" & '0' &  "00"  & '1' & '0' & "000" & '0' & "000"; -- SHL RA
        when "01100" => controlo <= "0111" & '0' &  "00"  & '1' & '0' & "000" & '0' & "000"; -- ADD RA, RB
        when "01101" => controlo <= "1000" & '0' &  "00"  & '1' & '0' & "000" & '0' & "000"; -- SUB RA, RB
        when "01110" => controlo <= "1001" & '0' &  "00"  & '0' & '0' & "000" & '1' & "000"; -- CMP RA, RB

        -- Saltos
        when "01111" => controlo <= "0000" & '0' &  "00"  & '0' & '0' & "011" & '0' & "000"; -- JZ RA, constante
        when "10000" => controlo <= "0000" & '0' &  "00"  & '0' & '0' & "100" & '0' & "000"; -- JN RA, constante
        when "10001" => controlo <= "0000" & '0' &  "00"  & '0' & '0' & "010" & '0' & "000"; -- JL constante
        when "10010" => controlo <= "0000" & '0' &  "00"  & '0' & '0' & "010" & '0' & "001"; -- JLE constante
        when "10011" => controlo <= "0000" & '0' &  "00"  & '0' & '0' & "010" & '0' & "010"; -- JE constante
        when "10100" => controlo <= "0000" & '0' &  "00"  & '0' & '0' & "010" & '0' & "011"; -- JGE constante
        when "10101" => controlo <= "0000" & '0' &  "00"  & '0' & '0' & "010" & '0' & "100"; -- JG constante
        when "10110" => controlo <= "0000" & '0' &  "00"  & '0' & '0' & "001" & '0' & "000"; -- JMP constante

        when others  => controlo <= "0000" & '0' &  "00"  & '0' & '0' & "000" & '0' & "000"; -- NOP
      end case;
    end process;

    SEL_ALU  <= controlo (15 downto 12);
    ESCR_P   <= controlo (11);
    SEL_DATA <= controlo (10 downto 9);
    ESCR_R   <= controlo (8);
    WR       <= controlo (7);
    SEL_PC   <= controlo (6 downto 4);
    ESCR_F   <= controlo (3);
    SEL_F    <= controlo (2 downto 0);

end Behavioral;
```

# MUX_PC:

```
----------------------------------------------------------------------------------
-- Company:
-- Engineer:
--
-- Create Date: 05.03.2026 15:19:30
-- Design Name:
-- Module Name: Mux_PC - Behavioral
-- Project Name:
-- Target Devices:
-- Tool Versions:
-- Description:
--
-- Dependencies:
```

```
--
-- Revision:
-- Revision 0.01 - File Created
-- Additional Comments:
--
----------------------------------------------------------------------------------


library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

entity MUX_PC is
   Port (
         S_FLAG : in std_logic;
         Operando1 : in std_logic_vector (7 downto 0);
         SEL_PC : in std_logic_vector (2 downto 0);
         ESCR_PC : out std_logic
        );
end MUX_PC;

architecture Behavioral of MUX_PC is
begin
   process(SEL_PC,S_FLAG,Operando1)
   begin
      case SEL_PC is
         when "000" => ESCR_PC <= '0';
         when "001" => ESCR_PC <= '1';
         when "010" => ESCR_PC <= S_FLAG;
         when "011" => ESCR_PC <= NOT(Operando1(0) or Operando1(1) or Operando1(2) or Operando1(3)
                        or Operando1(4) or Operando1(5) or Operando1(6) or Operando1(7));
         when "100" => ESCR_PC <= Operando1(7);
         when others => ESCR_PC <= '0';
      end case;
   end process;
end Behavior
```